

Festival

Nayra mängib festivalil mängu, mille peaauhinnaks on reis Laguna Coloradasse. Mängus ostetakse žetoonide eest kuponge. Kupongi ostmise võib žetoonide arvu suurendada. Nayra eesmärk on saada võimalikult palju kuponge.

Mängu alguses on tal A žetooni. Saadaval on N kupongi, mis on nummerdatud 0 kuni $N - 1$. Nayra peab kupongi i ostmiseks maksma $P[i]$ žetooni ($0 \leq i < N$) (ja tal peab enne ostu sooritamist olema vähemalt $P[i]$ žetooni). Iga kupongi saab osta ülimalt ühe korra.

Igal kupongil i ($0 \leq i < N$) on **tüüp** $T[i]$: täisarv, mis jääb **vahemikku 1 kuni 4, kaasa arvatud**. Pärast kupongi i ostmist korrutatakse Nayra allesjäänud žetoonide arv arvuga $T[i]$. See tähendab: kui tal on mingil hetkel X žetooni ja ta ostab kupongi i (milleks on vaja, et $X \geq P[i]$), siis on tal pärast ostu sooritamist $(X - P[i]) \cdot T[i]$ žetooni.

Sinu ülesanne on otsustada, milliseid kuponge ja millises järjekorras peaks Nayra ostma, et maksimeerida **kupongide** koguarvu, mis tal lõpuks on. Kui maksimaalset kupongide koguarvu on võimalik saavutada mitme erineva ostude järjestusega, võib tagastada ükskõik millise neist.

Realiseerimine

Sul tuleb kirjutada järgmine funktsioon:

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : Nayra esialgne žetoonide arv.
- P : vektor pikkusega N , mis kirjeldab kupongide hindu.
- T : vektor pikkusega N , mis kirjeldab kupongide tüüpe.
- Seda funktsiooni kutsutakse iga testi kohta välja täpselt üks kord.

Funktsioon peaks tagastama vektori R , mis määrab Nayra ostud järgmiselt:

- R pikkus peaks olema maksimaalne arv kuponge, mida ta saab osta.
- Vektori elemendid on nende kupongide indeksid, mida ta peaks ostma, kronoloogilises järjekorras. See tähendab, et ta ostab kõigepealt kupongi $R[0]$, seejärel kupongi $R[1]$ jne.
- Kõik R elemendid peavad olema omavahel erinevad.

Kui ühtegi kupongi ei saa osta, peaks R olema tühi vektor.

Sisendi piirangud

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ iga i korral, kus $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ iga i korral, kus $0 \leq i < N$.

Alamülesanded

Alamülesanne	Väärtus	Lisapiirangud
1	5	$T[i] = 1$ iga i korral, kus $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ iga i korral, kus $0 \leq i < N$.
3	12	$T[i] \leq 2$ iga i korral, kus $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra saab osta kõik N kupongi (mingis järjekorras).
6	16	$(A - P[i]) \cdot T[i] < A$ iga i puhul, kus $0 \leq i < N$.
7	18	Lisapiirangud puuduvad.

Näited

Näide 1

Vaatleme järgmist väljakutset:

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nayral on esialgu $A = 13$ žetooni. Ta saab osta 3 kupongi alljärgnevas järjekorras:

Ostetud kupong	Kupongi hind	Kupongi tüüp	Kupongide arv pärast ostu sooritamist
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Selles näites ei ole Nayral võimalik osta rohkem kui 3 kupongi ja ülalkirjeldatud ostude järjekord on ainus viis, kuidas ta 3 kupongi osta saab. Seega peaks funktsioon tagastama $[2, 3, 0]$.

Näide 2

Vaatleme järgmist väljakutset.

```
max_coupons(9, [6, 5], [2, 3])
```

Selles näites võib Nayra osta mõlemad kupongid mistahes järjekorras. Seega peaks funktsioon tagastama kas $[0, 1]$ või $[1, 0]$.

Näide 3

Vaatleme järgmist väljakutset.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Selles näites on Nayral üks žetoon, millest ei piisa ühegi kupongi ostmiseks. Seega peaks protseduur tagastama $[]$ (tühja vektori).

Näidishindaja

Sisendi vorming:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Väljundi vorming:

```
S
R[0] R[1] ... R[S-1]
```

Siin S on `max_coupons` poolt tagastatud vektori R pikkus.